

Environnement et anatomie des données

Analyse des données — Séance 1

Stefania Marcassa

CY Cergy Paris Université — L3 Économie

Où nous allons aujourd'hui

1. Le notebook : comment il fonctionne, et comment il trompe.
2. Les quelques objets Python dont nous aurons besoin tout le semestre.
3. Une première application économique : un panier de biens.
4. L'anatomie d'un tableau de données — la partie qui compte vraiment.

À la fin de la séance, vous devez pouvoir répondre à une question : **de quoi une ligne de ce fichier est-elle la description ?**

Le notebook

Tout le cours se fait dans le navigateur.

- Aucune installation, aucune version de Python à gérer
- Aucun problème de chemin d'accès
- Un compte Google suffit
- Le code s'exécute sur une machine distante, pas sur la vôtre

Le travail en local (Anaconda, VS Code) est possible mais **non assisté** : les TD ne sont pas consacrés au dépannage d'installations.

Premier notebook — maintenant

1. Connectez-vous à votre compte Google.
2. Allez sur `colab.research.google.com`
3. Cliquez sur **Nouveau notebook**.

Un notebook est une suite de **cellules** de deux types :

- les cellules de **code**, exécutées avec Maj + Entrée ;
- les cellules de **texte**, pour dire ce que l'on fait et pourquoi.

Un notebook sans cellules de texte est illisible — y compris pour vous, dans trois semaines.

Une première cellule

```
# Le taux de croissance entre deux années  
pib_2023 = 2822  
pib_2024 = 2891  
  
croissance = (pib_2024 - pib_2023) / pib_2023  
print(croissance)  
  
0.024450035435861798
```

Trois choses : le # introduit un commentaire, le = affecte une valeur, et rien ne s'affiche sans print().

Le piège de l'ordre d'exécution

Les cellules gardent leur effet même après modification. L'ordre dans lequel vous les avez exécutées n'est pas l'ordre dans lequel elles sont écrites.

```
tva = 0.20                # cellule exécutée en premier  
prix_ttc = 100 * (1 + tva)
```

```
tva = 0.055              # cellule ajoutée plus tard, plus haut
```

prix_ttc vaut toujours 120. Rien ne le signale.

Le réflexe à prendre

Avant de conclure quoi que ce soit : **Exécution** → **Tout exécuter**. Si le notebook ne passe pas de haut en bas, il est faux.

Un piège qui vous coûtera dix minutes

Lors du premier import de fichier, Colab peut renvoyer une erreur si les **cookies tiers** sont bloqués dans votre navigateur.

La correction :

1. Ouvrir les paramètres du navigateur
2. Autoriser `https://[*.]googleusercontent.com:443`

Notez-le maintenant. Vous en aurez besoin à la séance 2.

**Les objets dont nous aurons
besoin**

Quatre types de base

```
inflation = 2.4           # float : nombre a virgule
annee      = 2024         # int   : nombre entier
pays       = "France"    # str   : texte
zone_euro  = True         # bool  : vrai / faux

type(inflation)           # <class 'float'>
```

La fonction `type()` paraît inutile aujourd'hui. Elle sera votre premier réflexe de diagnostic à partir de la séance 2.

Pourquoi le type n'est pas un détail

Le code commune

Le code INSEE de Bourg-en-Bresse est 01053. Lu comme un nombre, il devient 1053 — et ne correspond plus à aucune commune lors d'une fusion de fichiers.

L'année

Une année stockée comme texte se trie dans l'ordre alphabétique. "1999" vient après "10000", et les comparaisons $>$ ne veulent plus rien dire.

La moitié des bugs de ce semestre viendra d'un type mal interprété à la lecture d'un fichier. Aucun ne produira de message d'erreur.

Listes : une suite ordonnée

```
prix = [1.20, 0.95, 2.40, 1.75]
```

```
prix[0]          # 1.2    -- on compte à partir de zero
prix[-1]         # 1.75   -- le dernier
prix[0:2]        # [1.2, 0.95] -- borne de droite exclue
len(prix)        # 4
sum(prix)        # 6.3
```

Deux conventions à intégrer une fois pour toutes : l'indexation commence à **zéro**, et la borne de droite d'une tranche est **exclue**.

Se tromper sur l'une ou l'autre ne produit pas d'erreur. Cela produit un résultat faux.

Dictionnaires : des paires clé-valeur

```
prix = {"pain": 1.20, "lait": 0.95, "cafe": 2.40}
```

```
prix["pain"]           # 1.2  
prix.keys()            # les biens  
prix.values()          # les prix  
"beurre" in prix       # False
```

Là où une liste retient un *ordre*, un dictionnaire retient une *correspondance*.

C'est la structure d'une ligne de tableau : chaque variable porte un nom, et à chaque nom correspond une valeur.

Lire un message d'erreur

```
prix["beurre"]
```

```
KeyError: 'beurre'
```

Un message d'erreur Python se lit **par la fin** : la dernière ligne dit ce qui ne va pas, celles du dessus disent où.

- `KeyError` — cette clé n'existe pas
- `NameError` — variable jamais définie (cellule non exécutée ?)
- `TypeError` — opération impossible entre ces types
- `FileNotFoundError` — le fichier n'est pas là où vous croyez

Une erreur qui s'affiche est une bonne nouvelle. Ce sont les calculs silencieusement faux qui coûtent cher.

Une première application

Deux dictionnaires suffisent à représenter le problème économique.

```
prix_2023 = {"pain": 1.20, "lait": 0.95, "cafe": 2.40}
```

```
prix_2024 = {"pain": 1.35, "lait": 1.02, "cafe": 2.28}
```

```
quantites_2023 = {"pain": 120, "lait": 90, "cafe": 24}
```

Notez que le prix du café *baisse*. Un indice de prix n'est pas la moyenne des hausses : c'est le coût d'un panier donné, comparé dans le temps.

Le coût du panier

```
depense_2023 = 0
for bien in quantites_2023:
    depense_2023 += prix_2023[bien] * quantites_2023[bien]

print(depense_2023)      # 287.1
```

Une boucle for parcourt les clés du dictionnaire; le += accumule. L'indentation n'est pas décorative : elle délimite le corps de la boucle.

Refaites le calcul avec prix_2024 et les *mêmes* quantités. Le rapport des deux dépenses est un indice de Laspeyres.

Nous venons de fixer les quantités à celles de 2023. Rien ne nous y obligeait.

Laspeyres

Quantités de l'année de base.

Combien coûterait aujourd'hui le panier d'hier ?

Paasche

Quantités de l'année courante.

Combien coûtait hier le panier d'aujourd'hui ?

Les deux mesurent « l'inflation ». Ils ne donnent pas le même chiffre, et l'écart n'est pas une erreur : les ménages substituent.

Anatomie d'un tableau de données

Trois mots à ne jamais confondre

id_menage	revenu	region
1001	32 400	Île-de-France
1002	28 900	Occitanie
1003	41 200	Bretagne

- Une **observation** : une ligne. Ici, un ménage.
- Une **variable** : une colonne. Ici, le revenu.
- L'**unité statistique** : ce que représente une ligne. Ici, le ménage — et non l'individu.

Pourquoi l'unité statistique est le premier piège

La même question économique change de réponse selon l'unité retenue.

Un exemple

« Quel est le revenu moyen ? »

Par **ménage** : un chiffre. Par **individu** : un autre. Par **unité de consommation** : un troisième.

Aucun n'est faux. Ils ne répondent simplement pas à la même question.

Avant toute analyse : **de quoi une ligne est-elle la description ?**

Trois structures de données

Coupe transversale

Plusieurs unités, une date.

Les salaires de 40 000 individus
en 2023.

Série temporelle

Une unité, plusieurs dates.

Le taux de chômage français,
1975–2025.

Panel

Plusieurs unités, plusieurs
dates.

Le PIB de 27 pays, 2000–2024.

La structure détermine ce que l'on peut estimer — et ce que l'on ne peut pas. Nous construirons un panel à la séance 4.

Pour chacun de ces jeux de données : quelle est l'unité statistique, et quelle est la structure ?

1. Le prix de clôture du CAC 40, chaque jour depuis 1990.
2. Les réponses à l'Enquête Emploi du deuxième trimestre 2024.
3. Les transactions immobilières enregistrées en France de 2014 à 2024.
4. Le taux d'emploi des 27 pays de l'UE, chaque année depuis 2000.

Le cas 3 est moins évident qu'il n'y paraît : discutons-en.

Pour la suite

1. Reprenez le calcul du panier et obtenez l'indice de Laspeyres pour 2024.
2. Vérifiez que votre notebook s'exécute entièrement de haut en bas.
3. Autorisez `googleusercontent.com` dans votre navigateur.
4. Récupérez le notebook de la séance 2 sur le site du cours.

`stefaniamarcassa.github.io/analyse_des_donnees`

Des questions ?